

MEDIATEK

MT8676 Yocto Camera Driver User Manual

Version: 1.1
Release date: 2024-10-15

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Jianmin Zhou	Official release
1.1	2024-10-15	Jianmin Zhou	Added multicam configuration info into Section 1.3.1.9 Configure static_ctx

Table of Contents

Version History 2

Table of Contents 3

1 Camera driver 4

1.1 Overview 4

1.2 Architecture/Process Overview 4

1.3 Configuration/Customization Guideline 4

1.3.1 How to Add a New Sensor 4

1.3.1.1 Porting File List 5

1.3.1.2 Modify Config Files 5

1.3.1.3 Add Sensor Files to Kernel 5

1.3.1.4 How to Add Sensor ID and Sensor Name 5

1.3.1.5 Modify dts Files 6

1.3.1.6 Add New Meta Files 8

1.3.1.7 Configure Sensor Format 8

1.3.1.8 Configure modestruct 8

1.3.1.9 Configure static_ctx 10

1.3.1.10 Configure VC Info 11

1.3.1.11 Configure Get Sensor ID Function 12

1.3.1.12 Configure Sensor Init Function 12

1.3.1.13 Configure get_sensor_usage Function 13

1.3.1.14 Sensor Mode Setting 15

1.3.1.15 Driver Functions 16

Exhibit 1 Terms and Conditions 23

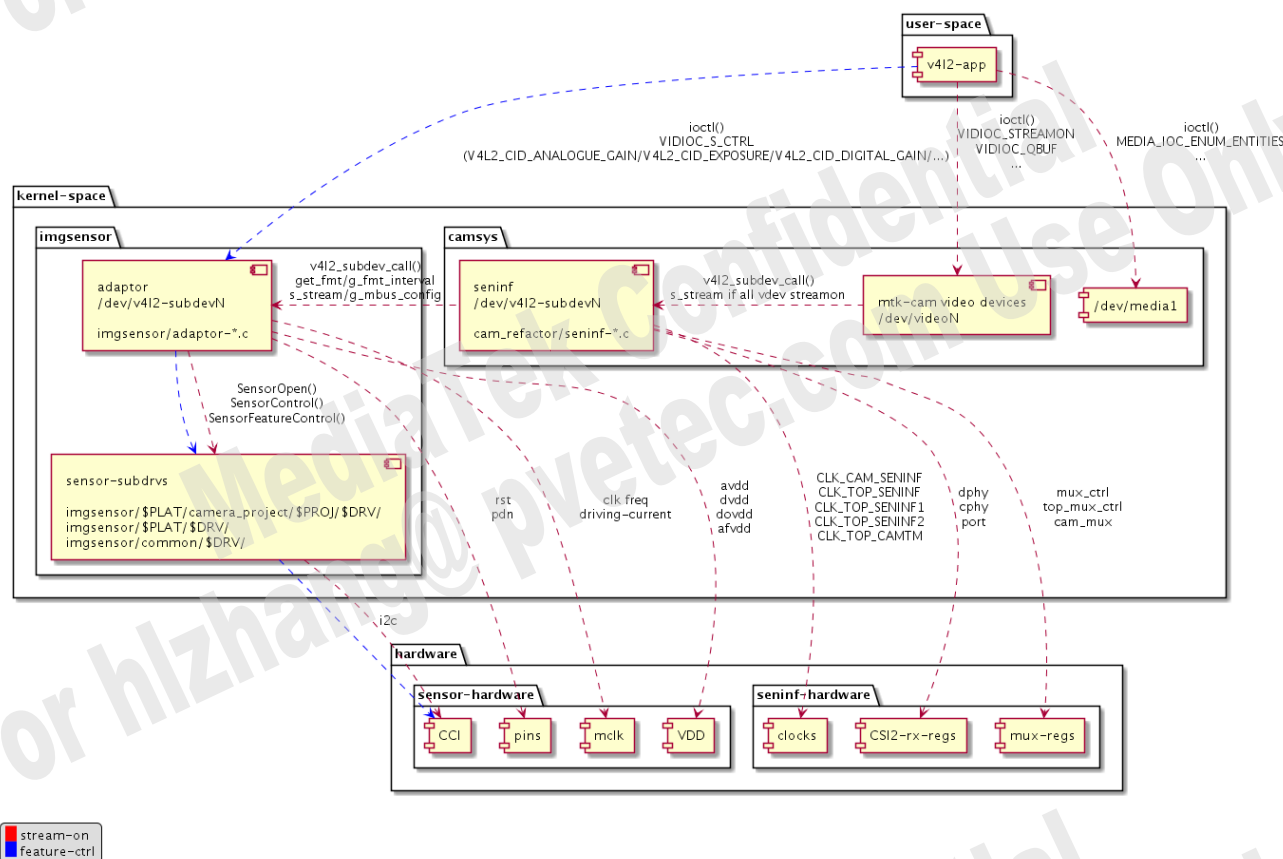
1 Camera driver

1.1 Overview

This section mainly introduces the MT8676 camera porting method and debugging methods and ideas for related issues.

1.2 Architecture/Process Overview

Camera driver is mainly divided into two parts: user space and kernel space. Both sides pass parameters through IOCTL. Raw sensor metadata, etc. are placed in user space; sensor related settings and power control are placed in kernel space.



1.3 Configuration/Customization Guideline

This section mainly introduces the Camera driver porting guide.

1.3.1 How to Add a New Sensor

This part is basically the same for YUV and raw sensor steps.

1.3.1.1 Porting File List

- **Config Files**
 - \src\kernel\linux\v6.1_mt8676\co_device_module\arch\arm64\configs\auto8676p1_64_defconfig
- **Kernel Space**
 - \src\kernel\linux\v6.1_mt8676\co_device_module\drivers\misc\mediatek\imgsensor\inc\kd_imgsensor.h
 - \src\kernel\linux\v6.1_mt8676\co_device_module\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi
 - \src\kernel\modules\mt8676\camera\imgsensor\src-v4l2\
- **User Space**
 - \src\multimedia\mtkcam-mt8676\mtkcam-utils\kernel-headers\mediatek\kd_imgsensor.h
 - \src\multimedia\camera-hal\mt8676\custom\common\hal\imgsensor_metadata\sensor\
 - \src\multimedia\camera-hal\mt8676\custom\mt6897\hal\imgsensor_metadata\

1.3.1.2 Modify Config Files

- \src\kernel\linux\v6.1_mt8676\co_device_module\arch\arm64\configs\auto8676p1_64_defconfig
 - CONFIG_CUSTOM_KERNEL_IMGSENSOR = "xxx_mipi_raw xxx_mipi_raw xxx_mipi_yuv"

Add the new sensor name above.

1.3.1.3 Add Sensor Files to Kernel

- \src\kernel\modules\mt8676\camera\imgsensor\src-v4l2\common\\${CamDrv}\

Refer to other sensors to add your own sensor driver directory in the above directory. For the naming format, refer to the figure below.

> vendor > mediatek > kernel_modules > mtkcam > imgsensor > src-v4l2 > common > max96712_mipi_yuv

Name	Date modified	Type	Size
 Makefile	3/11/2024 12:16 PM	File	1 KB
 max96712mipiyuv_Sensor	3/12/2024 3:05 AM	C Source	25 KB
 max96712mipiyuv_Sensor	3/11/2024 12:16 PM	H File	6 KB

1.3.1.4 How to Add Sensor ID and Sensor Name

- \src\multimedia\mtkcam-mt8676\mtkcam-utils\kernel-headers\mediatek\kd_imgsensor.h
- \src\kernel\linux\v6.1_mt8676\co_device_module\drivers\misc\mediatek\imgsensor\inc\kd_imgsensor.h

Make the following additions to the above two files.

- Add sensor ID

```
392 #define MAX96712MIPI_SENSOR_ID 0x52
```

- Add sensor name

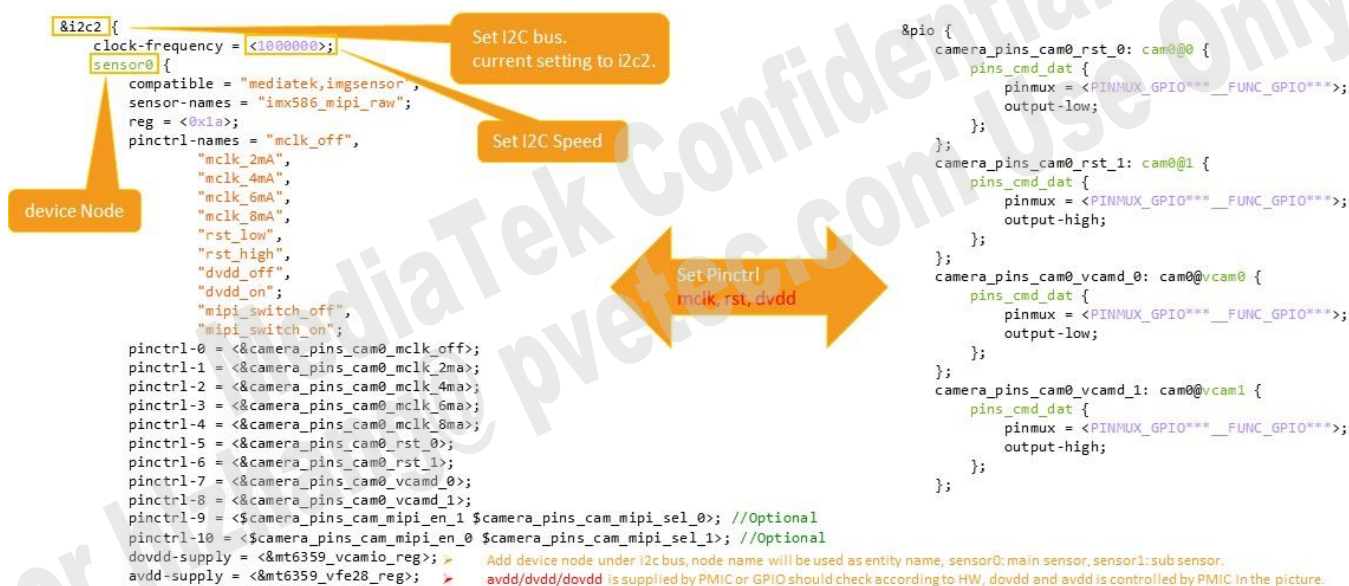
```
642 #define SENSOR_DRVNAME_MAX96712_MIPI_YUV "max96712_mipi_yuv"
```

1.3.1.5 Modify dts Files

\src\kernel\linux\v6.1_mt8676\co_device_module\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi

Find out which I2C the current sensor is connected to by analyzing the hardware schematic diagram, and then add the sensor configuration under the corresponding I2C, for example, as follows:

- Added sensor0



Regarding the relationship between seninf and sensor, you can refer to the figure below. The value of csi-port in seninf_top is the csi-port actually connected to the hardware.

Set mclk source & selectable frequency(6M~52M)

```

clocks = <topckgen_clk CLK_TOPCKGEN_UNIVPLL_192M_D32>,
<topckgen_clk CLK_TOPCKGEN_UNIVPLL_192M_D16>,
<topckgen_clk CLK_TOPCKGEN_F_F26M_CLK_D2>,
<topckgen_clk CLK_TOPCKGEN_UNIVPLL_192M_D10>,
<topckgen_clk CLK_TOPCKGEN_UNIVPLL_192M_D8>,
<topckgen_clk CLK_TOP_TCK_26M_MX9>,
① <topckgen_clk CLK_TOPCKGEN_UNIVPLL_D6_D8>,
<topckgen CLK_TOPCKGEN_CAMTG3_SEL>;
clock-names = "6", "12", "13", "24", "26", "48", "52", "mclk";
status = "okay";

port {
    sensor0_out: endpoint {
        remote-endpoint = <&seninf_csi_port_0_in>; ②
    };
};

```

Set endpoint

CLK_TOPCKGEN_CAMTG_SEL	mclk0
CLK_TOPCKGEN_CAMTG2_SEL	mclk1
CLK_TOPCKGEN_CAMTG3_SEL	mclk2
CLK_TOPCKGEN_CAMTG4_SEL	mclk3
.....	

```

&seninf_top {
    seninf_csi_port_0: seninf_csi_port_0 {
        compatible = "mediatek,seninf";
        csi-port = "0"; ③
        nvmem-cells = <csi_efuse0>;
        nvmem-cell-names = "rg_csi";
        port {
            seninf_csi_port_0_in: endpoint {
                remote-endpoint = <&sensor0_out>;
            };
        };
    };
};

```

1. CLK_TOPCKGEN_CAMTG3_SEL is mclk source, currently set to mclk2.
2. The port setting Check seninf's csi-port property, currently sensor0 connect to CSIO.
3. The setting of csi port represents which CSI the current sensor is connected to. For example, 0, 1, 2, 3, 0A, 0B, 1A, 1B etc.

The part corresponding to the configuration in dts is the power-on sequence as follows. This needs to be configured by referring to the power-on sequence in the sensor specification sheet.

1. Add power sequence info

- {power_type, voltage, delay_time in ms}
- Power on sequence from top to bottom
- Power off sequence from bottom to top
- mclk set frequency(24M) and drive current(4mA)

2. Add subdrv entry

- Naming rule: \$(folder-name)_entry
- Macro will generate sensor-list automatically

➤ Rename a copy according to the existing sensor driver source code, and the part of the register and sensor info should be confirmed with the vendor.

common/imx586_mipi_raw/imx586mipiraw_Sensor.c

```

static struct subdrv_pw_seq_entry pw_seq[] = {
    {HW_ID_MCLK, 24, 0},
    {HW_ID_RST, 0, 0},
    {HW_ID_AVDD, 2800000, 0},
    {HW_ID_AFVDD, 2800000, 0},
    {HW_ID_DVDD, 1100000, 0},
    {HW_ID_DOVDD, 1800000, 1},
    {HW_ID_MCLK_DRIVING_CURRENT, 4, 1},
    {HW_ID_RST, 1, 3},
};

const struct subdrv_entry imx586_mipi_raw_entry = {
    .name = "imx586_mipi_raw",
    .id = IMX586_SENSOR_ID,
    .pw_seq = pw_seq,
    .pw_seq_cnt = ARRAY_SIZE(pw_seq),
    .ops = &ops,
};

```

Take max96712 sensor as an example.

```

00842: static struct subdrv_pw_seq_entry pw_seq[] = {
00843:     {HW_ID_PDN, 0, 20},
00844:     {HW_ID_PDN, 1800000, 20},
00845: };
00846:
00847: const struct subdrv_entry max96712_mipi_yuv_entry = {
00848:     .name = "max96712_mipi_yuv",
00849:     .id = MAX96712_MIPI_SENSOR_ID,
00850:     .pw_seq = pw_seq,
00851:     .pw_seq_cnt = ARRAY_SIZE(pw_seq),
00852:     .ops = &ops,
00853: };

```

Power-on sequence

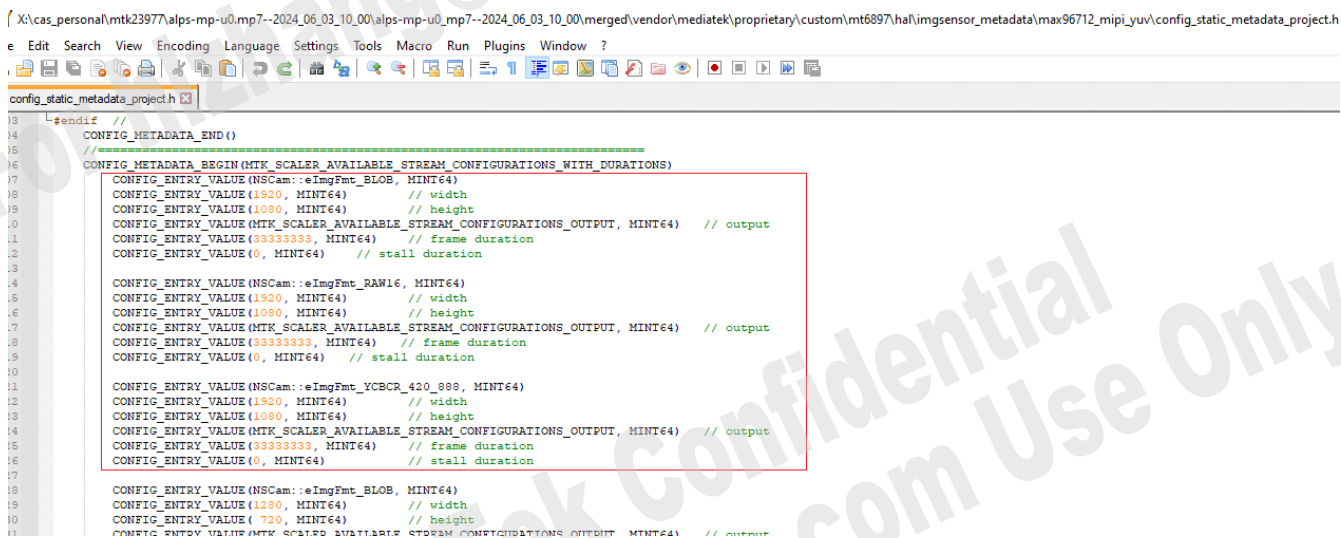
Modify sensor id and sensor name

1.3.1.6 Add New Meta Files

Refer to the max96712_mipi_yuv file to modify and add. You can copy it directly, and then modify the file name and sensor name in the file.

- \src\multimedia\camera-hal\mt8676\custom\common\hal\imgsensor_metadata\sensor\
- \src\multimedia\camera-hal\mt8676\custom\mt6897\hal\imgsensor_metadata\

If you need to add a new resolution, you can refer to the following code to add a new resolution.



```

13  #endif //
14  CONFIG_METADATA_END()
15  //
16  CONFIG_METADATA_BEGIN(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS)
17  CONFIG_ENTRY_VALUE(NSCam::eImgFmt_BLOB, MINT64)
18  CONFIG_ENTRY_VALUE(1920, MINT64) // width
19  CONFIG_ENTRY_VALUE(1080, MINT64) // height
20  CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
21  CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
22  CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
23
24  CONFIG_ENTRY_VALUE(NSCam::eImgFmt_RAW16, MINT64)
25  CONFIG_ENTRY_VALUE(1920, MINT64) // width
26  CONFIG_ENTRY_VALUE(1080, MINT64) // height
27  CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
28  CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
29  CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
30
31  CONFIG_ENTRY_VALUE(NSCam::eImgFmt_YCBCR_420_888, MINT64)
32  CONFIG_ENTRY_VALUE(1920, MINT64) // width
33  CONFIG_ENTRY_VALUE(1080, MINT64) // height
34  CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
35  CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
36  CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
37
38  CONFIG_ENTRY_VALUE(NSCam::eImgFmt_BLOB, MINT64)
39  CONFIG_ENTRY_VALUE(1280, MINT64) // width
40  CONFIG_ENTRY_VALUE(720, MINT64) // height
41  CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output

```

1.3.1.7 Configure Sensor Format

Configure UYVY or YUYV according to the format output by the bridge:

\src\kernel\modules\mt8676\camera\imgsensor\src-
v412\common\max96712_mipi_yuv\max96712mipiyuv_Sensor.c

Find all places where sensor_output_dataformat is configured and change it to the actual format.

.sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,

1.3.1.8 Configure modestruct

Confirm with the sensor vendor that the relevant parameters of the setting are filled in the structure.

Pclk->pixel clock, Unit Hz

Linlength->HTS

Framelength->VTS

linelength × framelength × fps= pclk

Grabwindow_width-> camera size width (There is no such value on mt8676. This value will be filled in in the solution position later.)

Grabwindow_height->camera size height (There is no such value on mt8676. This value will be filled in in the solution position later.)

Mipi_pixel_rate-> mipi datarate × lane_number/bitdepth, YUV bitdepth = 16

```

00344: static struct subdrv_mode_struct |node_struct[] = {
00345: {
00346:     .frame_desc = frame_desc_prev,
00347:     .num_entries = ARRAY_SIZE(frame_desc_prev),
00348:     .mode_setting_table = PARAM_UNDEFINED,
00349:     .mode_setting_len = PARAM_UNDEFINED,
00350:     .seamless_switch_group = PARAM_UNDEFINED,
00351:     .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
00352:     .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
00353:     .hdr_mode = HDR_NONE,
00354:     .raw_cnt = 1,
00355:     .exp_cnt = 1,
00356:     .pclk = 500000000,
00357:     .linelength = 8096,
00358:     .framelength = 1400,
00359:     .max_framerate = 300,
00360:     .mipi_pixel_rate = 500000000,
00361:     .readout_length = 0,
00362:     .read_margin = 10,
00363:     .framelength_step = 2,
00364:     .coarse_integ_step = 2,
00365:     .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
00366:     .multi_exposure_shutter_range[IMGSSENSOR_EXPOSURE_LE].min = 8,
00367:     .imgsensor_winsize_info = {
00368:         .full_w = SUPERFRAME_WIDTH,
00369:         .full_h = SUPERFRAME_HEIGHT,
00370:         .x0_offset = 0,
00371:         .y0_offset = 0,
00372:         .w0_size = SUPERFRAME_WIDTH,
00373:         .h0_size = SUPERFRAME_HEIGHT,
00374:         .scale_w = SUPERFRAME_WIDTH,
00375:         .scale_h = SUPERFRAME_HEIGHT,
00376:         .x1_offset = 0,
00377:         .y1_offset = 0,
00378:         .w1_size = SUPERFRAME_WIDTH,
00379:         .h1_size = SUPERFRAME_HEIGHT,
00380:         .x2_tg_offset = 0,
00381:         .y2_tg_offset = 0,
00382:         .w2_tg_size = SUPERFRAME_WIDTH,
00383:         .h2_tg_size = SUPERFRAME_HEIGHT,
00384:     },
00385:     .pdaf_cap = TRUE,
00386:     .imgsensor_pd_info = NULL,
00387:     .ae_binning_ratio = 1465,
00388:     .fine_integ_line = 826,
00389:     .delay_frame = 3,
00390:     .csi_param = {
00391:         .legacy_phy = 0,
00392:         .not_fixed_trail_settle = 1,
00393:         .not_fixed_dphy_settle = 1,
00394:         .dphy_trail = 0x4,
00395:         .dphy_data_settle = 0x20,
00396:         .cphy_settle = 13,
00397:         .clk_lane_no_initial_flow = 0,
00398:     },
00399:     .dpc_enabled = true, /* reg 0x0b06 */
00400: }, /* preview */

```

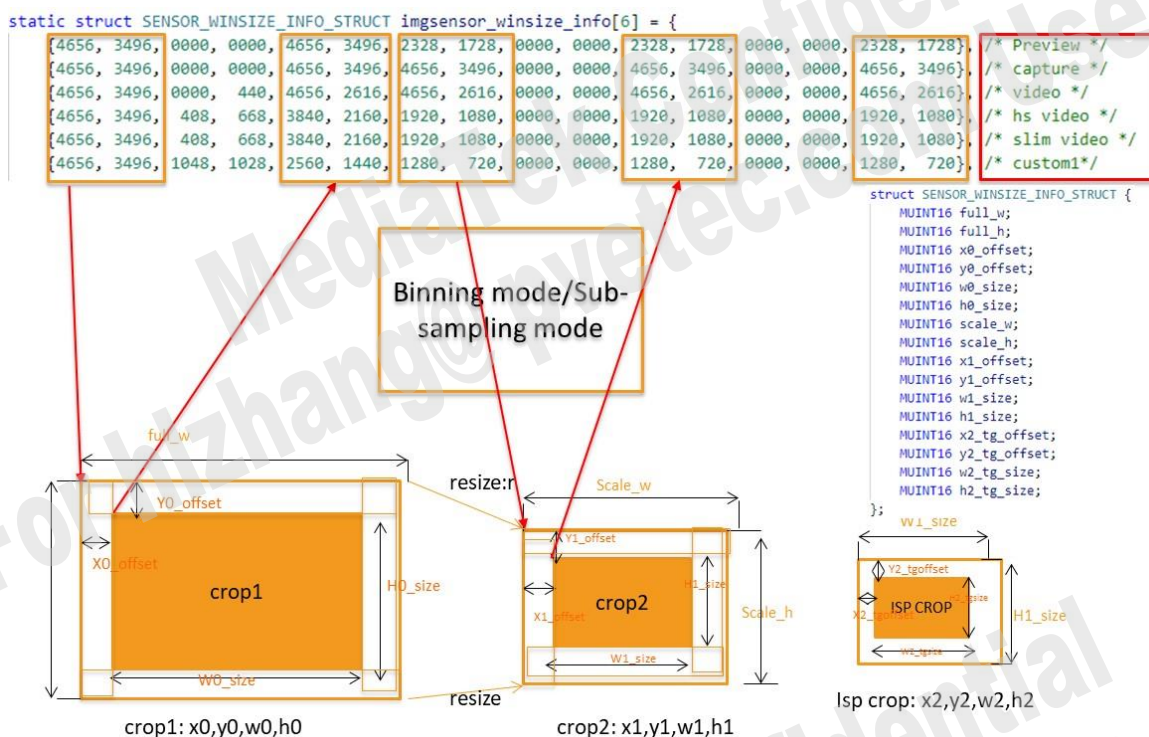
Configure Pclk, framelength, framerate, mipi picel rate

Configure format

Configure sinsize info

Configure hs trail & settle

The following figure borrows an explanation of winsize info from a previous version to describe the meaning of each field.



Limitation: full size (4:3) or full size (16:9) width and height need 4x alignment

Configure the sensor output size width and height, consistent with Grabwindow_width/Grabwindow_height size.

Grab window request:

1. The width must be a multiple of 16 and the height must be a multiple of 4.
2. The viewing angles are consistent at the same ratio; the ratios of 4:3 and 16:9 require that the window output by the sensor ensures that the horizontal direction is consistent with the full size.
3. The grab window setting recommendations are consistent with the window output by the sensor. In special cases: $\text{grabwindow width}(\text{height}) \leq \text{sensor output width}(\text{height}) - \text{startx}(y)$

1.3.1.9 Configure static_ctx

1. The sensor_id field below is filled with the corresponding sensor id.
2. The following i2c_add_table generally only configures 0x52, which is the address of the actual deserializer.
3. Configure resolution, here is the value of grabwindow mentioned earlier.
4. Configure mipi_lane_num, which represents the number of mipi lanes.
5. cam_type is filled in according to the type of the module scene. If it is a single sensor, you can fill in MTK_SENSOR_TYPE_SINGLE. It has the following values you can choose:
 - SENSOR_TYPE_COMB_AVM means combined frame AVM
 - SENSOR_TYPE_MUTI means multicam
 - SENSOR_TYPE_NON_COMB_AVM means non-combined frame avm

If it is configured as SENSOR_TYPE_MUTI, you also need to configure group_info as follow:

```

.group_info = {
    4,  master sensorId
    4,  vc num
    {
        {6,0,0}, slave sensorId, vc_id, link status
        {7,1,0},
        {8,2,0},
        {9,3,0},
    },
},
},

```

6. sensor_output_dataformat indicates the format of the data output by the module. Just fill in the actual format.

```

00571: static struct subdrv_static_ctx static_ctx = {
00572:     .sensor_id = MAX96712MIPI_SENSOR_ID,  Configure sensor id
00573:     .reg_addr_sensor_id = {0x0000},
00574:     .i2c_addr_table = {0x52, 0xFF},  Configure I2C addr
00575:     .i2c_burst_write_support = TRUE,
00576:     .i2c_transfer_data_type = I2C_DT_ADDR_16_DATA_8,
00577:     .eeprom_info = 0,
00578:     .eeprom_num = 0,
00579:     .resolution = {SUPERFRAME_WIDTH, SUPERFRAME_HEIGHT},  Configure resolution
00580:     .mirror = IMAGE_HV_MIRROR,
00581:
00582:     .mclk = 24,
00583:     .isp_driving_current = ISP_DRIVING_4MA,
00584:     .sensor_interface_type = SENSOR_INTERFACE_TYPE_MIPI,
00585:     .mipi_sensor_type = MIPI_OPHY_NCSI2,
00586:     .mipi_lane_num = SENSOR_MIPI_4_LANE,  Configure Lanes
00587:     .ob_pedestal = 0x40,
00588:     .cam_type = SENSOR_TYPE_NON_COMB_AVM,
00589:     .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
00590:     .sensor_data = PAGEGATH * 4

```

1.3.1.10 Configure VC Info

A single vc channel can be configured as follows:

```

00303:
00304: static struct mtk_mbus_frame_desc_entry frame_desc_prev[] = {
00305:     {
00306:         .bus.csi2 = {
00307:             .channel = 0,
00308:             .data_type = 0x1E, //yuv422
00309:             .hsize = FRAME_WIDTH,
00310:             .vsize = FRAME_HEIGHT,
00311:             .user_data_desc = VC_BRIDGE_RAW_0,
00312:         },
00313:     },
00314: };

```

If you have multiple VC channels, you can configure it as follows, mainly modifying channel and user_data_desc.

```

00304: static struct mtk_mbus_frame_desc_entry frame_desc_prev[] = {
00305:     {
00306:         .bus.csi2 = {
00307:             .channel = 0,
00308:             .data_type = 0x1E, //yuv422
00309:             .hsize = FRAME_WIDTH,
00310:             .vsize = FRAME_HEIGHT,
00311:             .user_data_desc = VC_BRIDGE_RAW_0,
00312:         },
00313:     },
00314:     {
00315:         .bus.csi2 = {
00316:             .channel = 1,
00317:             .data_type = 0x1E, //yuv422
00318:             .hsize = FRAME_WIDTH,
00319:             .vsize = FRAME_HEIGHT,
00320:             .user_data_desc = VC_BRIDGE_RAW_1,
00321:         },
00322:     },
00323:     {
00324:         .bus.csi2 = {
00325:             .channel = 2,
00326:             .data_type = 0x1E, //yuv422
00327:             .hsize = FRAME_WIDTH,
00328:             .vsize = FRAME_HEIGHT,
00329:             .user_data_desc = VC_BRIDGE_RAW_2,
00330:         },
00331:     },
00332:     {
00333:         .bus.csi2 = {
00334:             .channel = 3,
00335:             .data_type = 0x1E, //yuv422
00336:             .hsize = FRAME_WIDTH,
00337:             .vsize = FRAME_HEIGHT,
00338:             .user_data_desc = VC_BRIDGE_RAW_3,
00339:         },
00340:     },
00341: };

```

1.3.1.11 Configure Get Sensor ID Function

```

00249: static int return_sensor_id(struct subdrv_ctx *ctx)
00250: {
00251:     return read_cmos_sensor(ctx, MAX96712_ID_REG);
00252: }
00253:

```

1.3.1.12 Configure Sensor Init Function

The following function mainly configures the settings of the module, which needs to be debugged and generated by the customer and vendor.

```

88
89 static int max96712_sensor_init(struct subdrv_ctx *ctx)
90 {
91
92     DRV_LOG(ctx, "--MAX96712-ISX019-sensor_init start\n");
93     write_cmos_sensor(ctx, 0x0017, 0x14);
94     write_cmos_sensor(ctx, 0x0019, 0x94);
95     write_cmos_sensor(ctx, 0x0013, 0x75);

```

1.3.1.13 Configure get_sensor_usage Function

```

668
669 static int get_sensor_usage(struct subdrv_ctx *ctx, enum mtk_sensor_usage *usage)
670 {
671     *usage = MTK_SENSOR_USAGE_NONCOMB;
672     return 0;
673 }

```

Its value can be selected from the following. COMB represents combined frames AVM, MUTI represents multicam, NONCOMB represents uncombined frame AVM, Single sensor is configured as SINGLE.

```

67 enum mtk_sensor_usage {
68     MTK_SENSOR_USAGE_SINGLE,
69     MTK_SENSOR_USAGE_COMB,
70     MTK_SENSOR_USAGE_MUTI,
71     MTK_SENSOR_USAGE_NONCOMB,
72 };

```

1.3.1.13.1 Streaming Control

All initialization settings are placed in the sensor_init() function. MIPI enable and disable functions are placed in the streaming_control() function.

```

221
222 static int set_streaming_control(void *arg, bool enable)
223 {
224     struct subdrv_ctx *ctx = (struct subdrv_ctx *)arg;
225
226     DRV_LOG(ctx, "streaming_enable(0=Sw Standby, 1=streaming): %d\n", enable);
227     if (enable) {
228         /*turn on mipi*/
229         write_cmos_sensor(ctx, 0x040B, 0x42);
230         mdelay(5);
231     } else {
232         /*turn off mipi*/
233         write_cmos_sensor(ctx, 0x040B, 0x0);
234         mdelay(5);
235     }
236     return ERROR_NONE;
237 }
238 }

```

1.3.1.13.2 subdrv_static_ctx Structure Member Analysis

Member	Meaning
sensor_id	Sensor ID defined in kd_imgsensor.h.
reg_addr_sensor_id	Sensor register address where sensor ID is read. Up to 3 bytes.
i2c_addr_table	I2C write ID, end in 0xff, 4 for maximum. E.g., i2c_addr_table = {0x20, 0x6e, 0xff}, .
eeeprom_info	The address of eeeprom_info_struct. If there is no eeeprom device, set to 0.
eeeprom_num	Array size of the eeeprom_info_struct. If there is no eeeprom device, set to 0.
Resolution	Full pixel size of sensor output.
mirror	Set IMAGE_HV_MIRROR if the sensor orientation is 180 degrees difference from module orientation.
mclk	Overwrite MCLK frequency (MHz)
isp_driving_current	Overwrite MCLK driving current (mA)

Member	Meaning
sensor_interface_type	Data output interface type. Set default MIPI.
mipi_sensor_type	C-PHY or D-PHY.
mipi_lane_num	How many lanes/trios of MIPI-PHY.
Ob_pedestal	OB offset. General is 64.
sensor_output_dataformat	Bayer order, 4-cell order and SW/HW remo.
ana_gain_def	Default analog gain. Set 4x. 1024base.
ana_gain_min	Minimum analog gain. 1024base.
ana_gain_max	Maximum analog gain. 1024base.
ana_gain_type	Sony:type 0; OV:type 1; Samsung:type 2; Hynix:type 3; GC:type 4
ana_gain_step	Minimum valid step of analog gain. 1024base.
ana_gain_table	Use valid analog gain table. Remove unsuitable value which is not linearity. 1024base.
ana_gain_table_size	Size of analog gain table.
min_gain_iso	Set minimum ISO 100.
exposure_def	Default exposure line. Set 0x3D0.
exposure_min	Minimum exposure line.
exposure_max	Maximum exposure line.
exposure_step	Minimum valid step of exposure line.
exposure_margin	Maximum margin of exposure line.
frame_length_max	Maximum framelength
ae_effective_frame	AE effective frame
frame_time_delay_frame	The frame "frame length" setting takes effect. Sony sensor filled in 3; other sensors filled in 2.
start_exposure_offset	Parameter tuned in CTS sensor fusion test.
pdaf_type	Reference enum IMGSENSOR_PDAF_SUPPORT_TYPE_ENUM.
hdr_type	Reference enum IMGSENSOR_HDR_SUPPORT_TYPE_ENUM.
seamless_switch_support	If sensor supports seamless switch, set 1 to enable seamless switch function.
temperature_support	Set to 1 if sensor supports temperature sensor readout.
g_temp	Implement get temperature function.
g_gain2reg	Implement analog gain to register mapping rule.
s_gph	Implement set enable/disable group hold function.
s_cali	Implement the function to write data from eeprom_info_struct to sensor (e.g., QSC, cross-talk, ...)
reg_addr_stream	Sensor register address where streaming on/off is controlled.
Reg_addr_mirror_flip	Sensor register address where mirror/flip on/off is controlled. No use when set to 0.
reg_addr_exposure	Sensor register address where exposure line is set. Up to 3 channels of exposure. Up to 3 bytes.
long_exposure_support	Set to 1 if sensor supports long exposure left shift function.
reg_addr_exposure_lshift	Sensor register address where long exposure left shift is set.
reg_addr_ana_gain	Sensor register address where analog gain is set. Up to 3 channels of exposure. Up to 3 bytes.

Member	Meaning
reg_addr_frame_length	Sensor register address where framelength is set.
reg_addr_temp_en	If "temperature_support" is set to 1. Sensor register address where temperature sensor on/off is controlled.
reg_addr_temp_read	If "temperature_support" is set to 1. Sensor register address where temperature output is read.
reg_addr_auto_extend	For Sony sensor, sensor register address where auto extend function enable/disable is controlled.
reg_addr_frame_count	For Samsung sensor, sensor register address where frame count is read to determine whether sensor stream is off or not.
reg_addr_fast_mode	For Sony sensor, sensor register address where fast mode on/off is controlled.
init_setting_table	The address of sensor initial setting table
init_setting_len	Array size of the sensor initial setting table
mode	The address of subdrv_mode_struct
sensor_mode_num	Array size of the subdrv_mode_struct
list	The address of customized feature control list
list_len	Array size of the customized feature control list
checksum_value	The calculated value when Test Pattern output, for Camera Auto Test

1.3.1.14 Sensor Mode Setting

Subdrv_mode_struct mode_struct[] structure array is used to store the data configuration of different Sensor Modes.

The following uses preview mode as an example to illustrate some fields that need to be modified by the customer. For YUV sensor, generally the subsequent sensor mode and preview mode are exactly the same. There may be differences for the raw sensor, and you need to configure it according to actual needs.

```

00344: static struct subdrv_mode_struct {mode_struct[] = {
00345:     {
00346:         .frame_desc = frame_desc_prev,
00347:         .num_entries = ARRAY_SIZE(frame_desc_prev),
00348:         .mode_setting_table = PARAM_UNDEFINED,
00349:         .mode_setting_len = PARAM_UNDEFINED,
00350:         .seamless_switch_group = PARAM_UNDEFINED,
00351:         .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
00352:         .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
00353:         .hdr_mode = HDR_NONE,
00354:         .raw_cnt = 1,
00355:         .exp_cnt = 1,
00356:         .pclk = 500000000,
00357:         .linelength = 8096,
00358:         .framelength = 1400,
00359:         .max_framerate = 300,
00360:         .mipi_pixel_rate = 500000000,
00361:         .readout_length = 0,
00362:         .read_margin = 10,
00363:         .framelength_step = 2,
00364:         .coarse_integ_step = 2,
00365:         .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
00366:         .multi_exposure_shutter_range[IMGSSENSOR_EXPOSURE_LE].min = 8,
00367:         .imgsensor_winsize_info = {
00368:             .full_w = SUPERFRAME_WIDTH,
00369:             .full_h = SUPERFRAME_HEIGHT,
00370:             .x0_offset = 0,
00371:             .y0_offset = 0,
00372:             .w0_size = SUPERFRAME_WIDTH,
00373:             .h0_size = SUPERFRAME_HEIGHT,
00374:             .scale_w = SUPERFRAME_WIDTH,
00375:             .scale_h = SUPERFRAME_HEIGHT,
00376:             .x1_offset = 0,
00377:             .y1_offset = 0,
00378:             .w1_size = SUPERFRAME_WIDTH,
00379:             .h1_size = SUPERFRAME_HEIGHT,
00380:             .x2_tg_offset = 0,
00381:             .y2_tg_offset = 0,
00382:             .w2_tg_size = SUPERFRAME_WIDTH,
00383:             .h2_tg_size = SUPERFRAME_HEIGHT,
00384:         },
00385:         .pdaf_cap = TRUE,
00386:         .imgsensor_pd_info = NULL,
00387:         .ae_binning_ratio = 1465,
00388:         .fine_integ_line = 826,
00389:         .delay_frame = 3,
00390:         .csi_param = {
00391:             .legacy_phy = 0,
00392:             .not_fixed_trail_settle = 1,
00393:             .not_fixed_dphy_settle = 1,
00394:             .dphy_trail = 0x4,
00395:             .dphy_data_settle = 0x20,
00396:             .cphy_settle = 13,
00397:             .clk_lane_no_initial_flow = 0,
00398:         },
00399:         .dpc_enabled = true, /* reg 0x0b06 */
00400:     }, /* preview */

```

Configure Pclk, framelength, framerate, mipi pixel rate

Configure format

Configure sinsize info

Configure hs trail & settle

1.3.1.15 Driver Functions

1.3.1.15.1 Driver Operation Function List

The list of driver function is as follows:

```
789 static struct subdrv_ops ops = {
790     .get_id = get_imgsensor_id,
791     .init_ctx = init_ctx,
792     .open = open,
793     .get_info = common_get_info,
794     .get_resolution = common_get_resolution,
795     .control = common_control,
796     .feature_control = common_feature_control,
797     .close = common_close,
798     .get_frame_desc = common_get_frame_desc,
799     .get_temp = common_get_temp,
800     .get_csi_param = common_get_csi_param,
801     .vsync_notify = vsync_notify,
802     .update_sof_cnt = common_update_sof_cnt,
803     .parse_ebd_line = common_parse_ebd_line,
804     .get_sensor_usage = get_sensor_usage,
805 };
806
```

In the imgsensor code starting with the adapter name, it is often called into the driver code through subdrv_call, which is the function list above, such as:

```
ret = subdrv_call(ctx, get_id, &sensor_id);
```

The above call will call get_sensor_id in the function list above.

There are many functions starting with common in the above list. This part uses a common processing flow. No need to implement them specially in sensor driver.

The following is a description of the get_imgsensor_id and open functions that your company may change.

1.3.1.15.2 get_imgsensor_id Function

When the search sensor is turned on, the ID will be read through the get_imgsensor_id function. If the ID can be read successfully, the icon of the camera APP will be displayed on the UI.

The following functions are mainly used to obtain the sensor ID and determine whether there is a link to the module.

```

675 static int get_imgsensor_id(struct subdrv_ctx *ctx, u32 *sensor_id)
676 {
677     u8 i = 0;
678     u8 retry;
679
680     ctx->i2c_write_id = static_ctx.i2c_addr_table[0];
681     retry = 2;
682
683     while (static_ctx.i2c_addr_table[i] != 0xff) {
684         spin_lock(&imgsensors_drv_lock);
685         ctx->i2c_write_id = static_ctx.i2c_addr_table[i];
686         spin_unlock(&imgsensors_drv_lock);
687         do {
688             *sensor_id = return_sensor_id(ctx);
689             if (*sensor_id == (static_ctx.sensor_id)) {
690                 DRV_LOG(ctx, "i2c write id: 0x%x, sensor id: 0x%x\n",
691                     ctx->i2c_write_id, *sensor_id);
692                 return ERROR_NONE;
693             }
694             DRV_LOG(ctx, "Read id fail, id: 0x%x, i2c id: 0x%x\n",
695                 *sensor_id, ctx->i2c_write_id);
696             retry--;
697         } while (retry > 0);
698         i++;
699         retry = 2;
700     }
701
702     if (*sensor_id != (static_ctx.sensor_id)) {
703         *sensor_id = 0xFFFFFFFF;
704         return ERROR_SENSOR_CONNECT_FAIL;
705     }
706     return ERROR_NONE;
707 }

```

1.3.1.15.3 Open Function

It will be called every time when the camera is opened.

Read sensorid to confirm whether I2C communication is normal.

Call the sensor_init function to initialize the sensor registers.

```

688 static int open(struct subdrv_ctx *ctx)
689 {
690     u8 i = 0;
691     u8 retry = 2;
692     u16 sensor_id = 0;
693     u32 scenario_id = 0;
694
695     while (static_ctx.i2c_addr_table[i] != 0xff) {
696         spin_lock(&imgsensors_drv_lock);
697         ctx->i2c_write_id = static_ctx.i2c_addr_table[i];
698         spin_unlock(&imgsensors_drv_lock);
699         do {
700             sensor_id = return_sensor_id(ctx);
701             if (sensor_id == MAX96712ISX_SENSOR_ID) {
702                 DRV_LOG(ctx, "i2c write id: 0x%x, sensor id: 0x%x\n",
703                     ctx->i2c_write_id, sensor_id);
704                 break;
705             }
706             DRV_LOG(ctx, "Read id fail, id: 0x%x, i2c id: 0x%x\n",
707                 sensor_id, ctx->i2c_write_id);
708             retry--;
709         } while (retry > 0);
710         i++;
711         if (sensor_id == MAX96712ISX_SENSOR_ID)
712             break;
713         retry = 2;
714     }
715     if (sensor_id != MAX96712ISX_SENSOR_ID)
716         return -ERROR_SENSOR_CONNECT_FAIL;
717
718     /* initail sequence write in */
719
720     max96712_sensor_init(ctx);
721     retry = 3;
722     do {
723         if (Chenck_Sensor_Signal(ctx)) {
724             DRV_LOG(ctx, "Sensor Not Connect");
725             mdelay(5);
726             retry--;
727         } else
728             break;
729     } while (retry > 0);
730
731     if (retry == 0) {
732         DRV_LOG(ctx, "OPEN FAIL!Sensor Not Connect");
733         return -ERROR_SENSOR_CONNECT_FAIL;
734     }
735 }

```

```

736 » /* move camera init to initctx(), void max9286 init too much time */
737 »
738 » spin_lock(&imgsensor_drv_lock);
739 »
740 » memset(ctx->exposure, 0, sizeof(ctx->exposure));
741 » memset(ctx->ana_gain, 0, sizeof(ctx->gain));
742 » ctx->exposure[0] = ctx->s_ctx.exposure_def;
743 » ctx->ana_gain[0] = ctx->s_ctx.ana_gain_def;
744 » ctx->current_scenario_id = scenario_id;
745 » ctx->pclk = mode_struct[scenario_id].pclk;
746 » ctx->line_length = mode_struct[scenario_id].line_length;
747 » ctx->frame_length = mode_struct[scenario_id].frame_length;
748 » ctx->frame_length_rg = ctx->frame_length;
749 » ctx->current_fps = ctx->pclk / ctx->line_length * 10 / ctx->frame_length;
750 » ctx->readout_length = mode_struct[scenario_id].readout_length;
751 » ctx->read_margin = mode_struct[scenario_id].read_margin;
752 » ctx->min_frame_length = ctx->frame_length;
753 » ctx->autoflicker_en = FALSE;
754 » ctx->test_pattern = 0;
755 » ctx->ihdr_mode = 0;
756 » ctx->pdaf_mode = 0;
757 » ctx->hdr_mode = 0;
758 » ctx->extend_frame_length_en = 0;
759 » ctx->is_seamless = 0;
760 » ctx->fast_mode_on = 0;
761 » ctx->sof_cnt = 0;
762 » ctx->ref_sof_cnt = 0;
763 » ctx->is_streaming = 0;
764 » spin_unlock(&imgsensor_drv_lock);
765 »
766 » return ERROR_NONE;
767 }

```

Remark:

1. A note about resolution

Because for different scenarios, the size of the image given to user space is different from the size of the actual VC channel charged by seninf.

Therefore, there will be two resolutions in the driver. Customers can take max96712, the uncombined frame avm driver, as an example.

Search for the following macros to compare configurations.

```

00026: #define VC 0
00027:
00028: #define SUPERFRAME_WIDTH 7680
00029: #define SUPERFRAME_HEIGHT 1280
00030:
00031: #define FRAME_WIDTH 1920
00032: #define FRAME_HEIGHT 1280
00033:
00034: #define MAX96712_ID_REG 0x00U
00035: #define SC22AT_I2C_ADDR 0x60
00036: #define MAX96717_I2C_ADDR 0x80
00037: #define MAX96712_SENSOR_ID 0x52
00038:

```

This is the maximum resolution seen by the upper layer.
If it is a non-combined frame AVM, the width here will be 7680
single VC channel resolution

2. Regarding the GPIO power supply part in dts, you can also choose the regulator packaging method.

Usage is as follows:

```

gpio141_regulator: gpio-141 {
    compatible = "regulator-fixed";
    regulator-name = "gpio141_regulator";
    regulator-min-microvolt = <1800000>;
    regulator-max-microvolt = <1800000>;
    gpio = <&pio 141 0x0>;
    enable-active-high;
    status = "okay";
};

```

```

sensor0: sensor0@52 {
    compatible = "mediatek,imgsensorsensor";
    sensor-names = "max96712_mipi_yuv";
    reg = <0x52>;
    #thermal-sensor-cells = <0>;
    pdn-supply = <&gpio141_regulator>;

    clocks = <&topckgen_clk CLK_TOP_UNIVPLL_192M_D32>,
        <&topckgen_clk CLK_TOP_UNIVPLL_192M_D16>,
        <&topckgen_clk CLK_TOP_F26M_CLK_D2>,
        <&topckgen_clk CLK_TOP_UNIVPLL_192M_D10>,
        <&topckgen_clk CLK_TOP_UNIVPLL_192M_D8>,
        <&topckgen_clk CLK_TOP_UNIVPLL_D6_D16>,
        <&topckgen_clk CLK_TOP_UNIVPLL_D6_D8>,
        <&topckgen_clk CLK_TOP_CAMTG5_SEL>;
    clock-names = "6", "12", "13", "19.2", "24", "26", "52", "mclk";

    status = "okay";

    port {
        sensor0_out: endpoint {
            remote-endpoint = <&seninf_csi_port_2_in>;
        };
    };
};

```

UT test command

C:\Users>adb shell sentest_v4l2 //just search sensor, not streaming

[show_Sensors]sensorNum 2

[show_Sensors]name:SENSOR_DRVNAME_OV05A20_MIPI_RAW type:0

[show_Sensors]index:0, SensorDevIdx:1

[show_Sensors]name:SENSOR_DRVNAME_IMX576_MIPI_RAW type:0

[show_Sensors]index:1, SensorDevIdx:2

[main]Param: 1 <sensorDev> <scenario> <fps>

[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)

[main]<scenario> : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)

show that sensorNum is not 0, and display the specific name of the sensor, that is, the search sensor is successful, Then test the output of the sensor.

C:\Users>adb shell sentest_v4l2 1 0 //search sensor, then open main (1) sensor preview (0)

```

[test_SensorInterface]name:SENSOR_DRVNAME_MAX96712ISX_MIPI_YUV type:1
[test_SensorInterface]index:0, SensorDevIdx:1
[test_SensorInterface]scenario(0) , binning type = 1465
[test_SensorInterface]scenario(0) , rawFmtType = 0
[test_SensorInterface]scenario(0) , OB_pedestal = 64
[test_SensorInterface]scenario(0) , sensorFormatOrder = 4
[test_SensorInterface]scenario(0) , seamless targets count = 0
[test_SensorInterface]Sensor index:0, scenarioId:0
[test_SensorInterface]crop w:1280, h:720
[test_SensorInterface]HDR_Support:0
[test_SensorInterface]scenario(0) , RGBW_Support = 0
[test_SensorInterface]scenario(0) , PDAF_Support = 0, dpc(1), pdc(0)
[test_SensorInterface]scenario(0) , Def_MirrorFlip = 3
[test_SensorInterface]scenario(0) , sensorFormatCellType = 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_CROP_WIN_INFO ret 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_RGBW_OUTPUT_MODE rgbw mode = 0
[test_SensorInterface]ana gain min/max = 1024/65536
[test_SensorInterface]dig gain min/max = 1024/16384
[test_SensorInterface]dig gain step = 4
[test_SensorInterface]SENSOR_CMD_GET_OFFSET_TO_START_OF_EXPOSURE exposure_offset = 500000
[test_SensorInterface]phalSensor->powerOn start
[test_CID] [test g volatile_ctrl] scenario (by set format)
[test_CID] [V4L2_CID_VBLANK] ret:0 VB:680
[test_CID] [V4L2_CID_HBLANK] ret:0 HB:4434
[test_CID] [V4L2_CID_MTK_SENSOR_PIXEL_RATE] ret:0 pixel rate:200000000
[test_CID] [V4L2_CID_MTK_STAGGER_INFO] ret:0 count:0 order:
[test_CID] [V4L2_CID_MAX_EXP_TIME]ret 0 exposure 0, max_exp_time 0
[test_CID] [V4L2_CID_MAX_EXP_TIME]ret 0 exposure 1, max_exp_time 0
[test_CID] [V4L2_CID_MAX_EXP_TIME]ret 0 exposure 2, max_exp_time 0
[streamon]sensor: idx=0 devpath=/dev/v4l-subdev26 drvname=max96712isx_mipi_yuv
[streamon]seninf: devpath=/dev/v4l-subdev4 cid=0x98f901 en=1
[test_SensorInterface]main/rolling_shutter=20571428

[test_SensorInterface]main/exp_margin=1371

[test_SensorInterface]main/atv-Continuous=1

[test_SensorInterface]main/atv-BypassSensorScenario=0

[test_SensorInterface]main/min_shutter=24 shutter_step=2

[test_SensorInterface]TgInfo[0] = 0
[test_SensorInterface]pixelMode[0] = 0
[test_SensorInterface]mipi rate = 200000000
[test_SensorInterface]wait enter key to end sensorIndex:0..

```

Check the seninf status command as follows. After finding the result, you can send it to MediaTek for confirmation.

CMD:

```
cat /sys/devices/platform/soc/1a00e000.seninf-top/status
```

```

255|auto8676p1_64_bsp:/ # cat /sys/devices/platform/soc/1a00e000.seninf-top/status
clk-fmeter-isp(687576) clk-fmeter-csi0(26000) clk-fmeter-csi1(0) clk-fmeter-csi2(0) clk-fmeter-csi3(312000) clk-fmeter-csi4(0) clk-fmeter-csi5(0)

[seninf-8] port 3 intf 6 test 0 cphy 0 lanes 0
[seninf-7] port 3 intf 6 test 0 cphy 0 lanes 0
source sensor7 flags 0x4

[seninf-6] port 3 intf 6 test 0 cphy 0 lanes 4
source sensor6 flags 0x5
csirx_mac_csi irq_stat 0x003f0324, seninf_irq_stat 0x00000000
csi2_line_frame_num 0x00000000
csi2_packet_status 0x12c0012b
csi2_packet_cnt_status 0x000004fb
rx-ana_settle_ck 0x00 dt 0x1a
rx-ana_trail_en 0 param 0x00
CSIRX_MAC_CSI2_RESYNC_MERGE_CTRL 0x00000106 CDPHY_RX_ANA_SETTING_1 0x04340244 DPHY_RX_SPARE0 0x000000f1
data_not_enough_cnt : <0>
err_lane_resync_cnt : <0>
crc_err_cnt : <0>
ecc_err_double_cnt : <0>
ecc_err_corrected_cnt : <0>
after write clear csi_irq 0x0 seninf_irq_stat 0x00000000
[0] vc 0x0 dt 0x2b mux 1 cam 0
mux[1] en 1 src 6 irq_stat 0xc
fifo_overrun_cnt : <0>
cam[0] en 1 Src 8 exp 0x8700f00 res 0x8700f00 err 0x0 irq_stat 0x100
size_err_cnt : <0>
--- mux meter ---
width 3840 height 2160
h_valid 0, h_blank 0
v_valid 0, v_blank 0
mipi_pixel_rate 0
v_blank 0 us
h_blank 0 us
line_time 0 us

[seninf-2] port 2 intf 4 test 0 cphy 0 lanes 0
[seninf-1] port 1 intf 2 test 0 cphy 0 lanes 0
auto8676p1_64_bsp:/ #

```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.